

Національний технічний університет України
«Київський політехнічний інститут» ім. Ігоря Сікорського
Факультет прикладної математики
Кафедра системного програмування і спеціалізованих комп’ютерних систем

Лабораторна робота №1

з дисципліни «Розробка програмного забезпечення»

Виконав студент групи: КВ-34
Чижов Владислав

Київ 2026

1. Тема та мета роботи
Тема: Вступ до ROS. Вузли.

Мета роботи: навчитися працювати з ROS 2 у Docker-контейнері, створювати та збирати C++ пакети за допомогою colcon, а також перевірити роботу вузла-компонента і керованих вузлів Lifecycle Nodes.

2. Програмне забезпечення
Docker Desktop, ROS 2 Jazzy, Visual Studio Code, C++, colcon, CMake, бібліотеки rclcpp, rclcpp_components та rclcpp_lifecycle.

3. Хід виконання лабораторної роботи

3.1. Завдання 1.1. Налаштування середовища та робочого простору
Спочатку я налаштував Docker Desktop та перевірив його запуск на контейнері hello-world. Для лабораторної використав образ ROS 2 Jazzy, створив Dockerfile та скрипт entrypoint.sh, який підключає середовище ROS 2. Після цього зробив робочу область ros2_ws/src і пакет lab1_nodes. Проєкт збирав командою colcon build --symlink-install, а після збирання виконав source install/setup.bash.

Для перевірки використав команди: docker run hello-world, echo $ROS_DISTRO, ros2 --help, colcon build --symlink-install та ros2 pkg list | grep lab1_nodes.

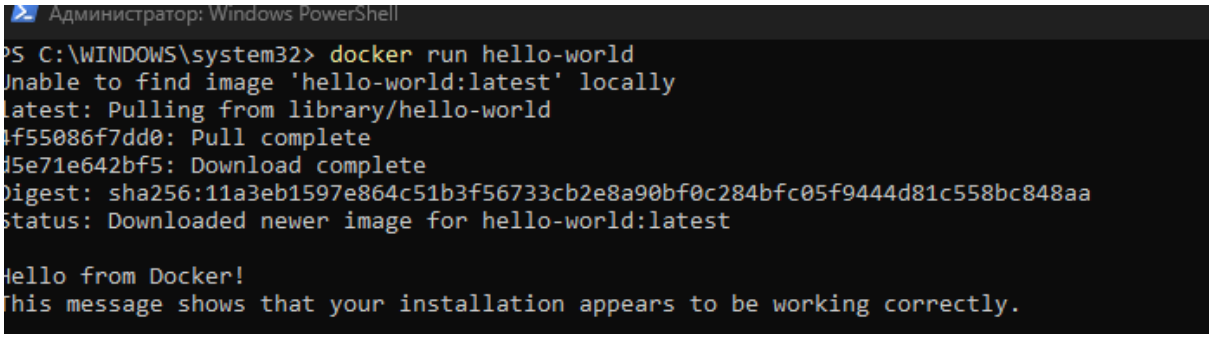


Рисунок 1 – Перевірка Docker Desktop і запуску hello-world

```
root@85b7a9401b3a:/workspace# echo $ROS_DISTRO
jazzy
root@85b7a9401b3a:/workspace# ros2 --help
usage: ros2 [-h] [--use-python-default-buffering]
            Call `ros2 <command> -h` for more
            detailed usage. ...

ros2 is an extensible command-line tool for ROS 2.

options:
  -h, --help            show this help message
                        and exit
  --use-python-default-buffering
                        Do not force line
                        buffering in stdout and
                        instead use the python
                        default buffering, which
                        might be affected by
                        PYTHONUNBUFFERED/-u and
                        depends on whatever
                        stdout is interactive or
                        not

Commands:
  action     Various action related sub-commands
  bag        Various rosbag related sub-commands
  component  Various component related sub-commands
  daemon     Various daemon related sub-commands
  doctor     Check ROS setup and other potential issues
  interface  Show information about ROS interfaces
  launch     Run a launch file
  lifecycle  Various lifecycle related sub-commands
  multicast  Various multicast related sub-commands
  node       Various node related sub-commands
  param      Various param related sub-commands
  pkg        Various package related sub-commands
  plugin     Various plugin related sub-commands
  run        Run a package specific executable
  security   Various security related sub-commands
  service    Various service related sub-commands
  topic      Various topic related sub-commands
  wtf        Use `wtf` as alias to `doctor`
```

Рисунок 2 – Робота ROS 2 в контейнері (echo $ROS_DISTRO)

```
oot@85b7a9401b3a:/workspace/ros2_ws# colcon build --symlink-install
tarting >>> lab1_nodes
inished <<< lab1_nodes [5.42s]

ummary: 1 package finished [5.66s]
oot@85b7a9401b3a:/workspace/ros2_ws# source install/setup.bash
oot@85b7a9401b3a:/workspace/ros2_ws# ros2 pkg list | grep lab1_nodes
ab1_nodes
oot@85b7a9401b3a:/workspace/ros2_ws#
```

Рисунок 3 – Успішне збирання пакета засобом colcon

Фактичний результат: Docker успішно запустив тестовий контейнер hello-world. У контейнері визначається ROS 2 Jazzy, а пакет lab1_nodes зібрався без помилок. Після підключення install/setup.bash назва пакета з’явилася у списку ROS 2.

3.2. Завдання 1.2. Розробка вузла-компонента

Для другого завдання створив C++ клас DemoComponent у просторі імен lab1_nodes. Клас успадковується від rclcpp::Node, а його конструктор приймає const rclcpp::NodeOptions &. Для реєстрації використав макрос RCLCPP_COMPONENTS_REGISTER_NODE. У CMakeLists.txt налаштував збирання бібліотеки SHARED та її реєстрацію через rclcpp_components_register_nodes.

Перевірка: запустив ros2 run rclcpp_components component_container, а потім завантажив компонент командою ros2 component load /ComponentManager lab1_nodes lab1_nodes::DemoComponent. Результат перевірив командами ros2 component list та ros2 node list.

```
root@85b7a9401b3a:/workspace# ros2 component load /ComponentManager lab1_nodes lab1_nodes::DemoComponent
Loaded component 1 into '/ComponentManager' container node as '/demo_component'
root@85b7a9401b3a:/workspace# ros2 component list
/ComponentManager
  1  /demo_component
root@85b7a9401b3a:/workspace# ros2 node list
/ComponentManager
/demo_component
root@85b7a9401b3a:/workspace#
```

Рисунок 4 – Реєстрація та успішне завантаження компонента

Фактичний результат: компонент завантажився у ComponentManager, після чого в переліку компонентів з’явився demo_component. Команда ros2 node list також показала вузол /demo_component.

3.3. Завдання 1.3. Керовані вузли Lifecycle Nodes

Для останнього завдання написав клас DemoLifecycleNode на основі rclcpp_lifecycle::LifecycleNode. У ньому реалізував обробники основних переходів станів. На базі цього класу запустив два окремі вузли — managed_one і managed_two. Їхні стани змінював через команди ros2 lifecycle set, а результати перевіряв через ros2 lifecycle get.

Для перевірки використав ros2 node list, ros2 lifecycle get /managed_one, ros2 lifecycle set /managed_one configure та activate. Такі самі переходи виконав для /managed_two. Для першого вузла додатково перевірив deactivate і cleanup.

```
PS D:\kpi\rpz\lab1> docker exec -it ros2_lab1 bash
root@85b7a9401b3a:/workspace# source /opt/ros/jazzy/setup.bash
root@85b7a9401b3a:/workspace# source /workspace/ros2_ws/install/setup.ba
sh
root@85b7a9401b3a:/workspace# ros2 node list
/managed_one
/managed_two
root@85b7a9401b3a:/workspace#
```

Рисунок 5 – Два запущені керовані вузли

```
root@85b7a9401b3a:/workspace# ros2 lifecycle get /managed_one
unconfigured [1]
root@85b7a9401b3a:/workspace# ros2 lifecycle get /managed_two
unconfigured [1]
root@85b7a9401b3a:/workspace# ros2 lifecycle set /managed_one configure
Transitioning successful
root@85b7a9401b3a:/workspace# ros2 lifecycle get /managed_one
inactive [2]
root@85b7a9401b3a:/workspace# ros2 lifecycle set /managed_one activate
Transitioning successful
root@85b7a9401b3a:/workspace# ros2 lifecycle get /managed_one
active [3]
root@85b7a9401b3a:/workspace# ros2 lifecycle set /managed_two configure
Transitioning successful
root@85b7a9401b3a:/workspace# ros2 lifecycle set /managed_two activate
Transitioning successful
root@85b7a9401b3a:/workspace# ros2 lifecycle get /managed_one
active [3]
root@85b7a9401b3a:/workspace# ros2 lifecycle get /managed_two
active [3]
root@85b7a9401b3a:/workspace#
```

Рисунок 6 – Переведення обох вузлів у стан Active

```
root@85b7a9401b3a:/workspace# ros2 lifecycle set /managed_one deactivate
Transitioning successful
root@85b7a9401b3a:/workspace# ros2 lifecycle get /managed_one
inactive [2]
root@85b7a9401b3a:/workspace# ros2 lifecycle set /managed_one cleanup
Transitioning successful
root@85b7a9401b3a:/workspace# ros2 lifecycle get /managed_one
unconfigured [1]
root@85b7a9401b3a:/workspace#
```

Рисунок 7 – Деактивація та очищення конфігурації вузла

Фактичний результат: ROS 2 бачить обидва вузли. Після configure вони перейшли у стан Inactive, а після activate — у стан Active (active [3]). Для managed_one також успішно виконав deactivate і cleanup та повернув його у стан Unconfigured (unconfigured [1]).

5. Висновки

Під час цієї лабораторної я вперше на практиці розібрався, як працює ROS 2 у Docker. На початку були труднощі з налаштуванням WSL у Windows, але після цього вдалося запустити контейнер і створити робоче середовище. Далі я зібрав пакет lab1_nodes через colcon та перевірив, що ROS 2 його бачить.

Також я створив C++ компонент і завантажив його у ComponentManager. Окремо перевірив роботу двох Lifecycle-вузлів: зміг налаштувати й активувати їх, а для одного з них виконати деактивацію та очищення конфігурації. Завдяки цьому стало зрозуміліше, чим звичайний компонент відрізняється від вузла з керованим життєвим циклом.

Найкориснішим для мене було побачити, що стани вузлів можна змінювати командами з іншого термінала, не переписуючи програму. Отримані навички можна використати в наступних лабораторних роботах із ROS 2.

6. Посилання на вихідний код

Git-репозиторій з вихідним кодом: https://github.com/mehnn/ROS2_Lab1